

Update: A Claimed $\mathcal{O}(2^{0.49n})$ Algorithm for Subset Sum

Le Duc Minh

October 9, 2026

Abstract

The accompanying survey, *A Survey of Exact Algorithms for Subset Sum and Its Variants*, described the fixed-exponent worst-case improvement over $2^{n/2}$ as open. On October 6, 2026, OpenAI released a manuscript collection containing a preprint dated October 4 that claims a uniform randomized classical $\mathcal{O}(2^{0.49n})$ algorithm for Subset Sum. This note records the exact scope of that claim, its proof pipeline, its current verification status, and the resulting changes to the research program in this repository. It is an audit update, not an independent proof of the claimed theorem.

1 Why this update is separate

The earlier survey remains useful as a record of the preprint landscape it described, but several of its central statements are now historically dated. In particular, its introduction, result table, chronology, conclusion, and open-problem discussion say that ordinary worst-case Subset Sum had no known constant improvement below $2^{n/2}$. Rewriting each statement in place would hide the distinction between the prior landscape and a new, not yet independently checked claim. This dated note supplies the correction and the appropriate qualification.

2 The claimed result

The primary artifact is the preprint *Subset Sum in Time $\mathcal{O}(2^{0.49n})$* [1], included in OpenAI's mathematics collection [2]. Its Theorem 1.1 claims a single uniform randomized classical word-RAM algorithm with the following guarantees.

- The input is a target $t \geq 0$ and positive integer weights $a_1, \dots, a_n$, with repeated weights and the empty subset allowed.
- Let b be the largest input bit length. The word length is $w = \lceil 4(n + b + \log_2(n + 2)) \rceil$ bits. Random word generation, input access, preprocessing, and writable storage all count toward the running time.
- The algorithm halts on every random execution and returns the correct decision answer with probability at least $2/3$ for every fixed input.
- For every fixed $c > 0$, inputs with $b \leq n^c$ are processed in $C_c 2^{0.49n}$ word operations for sufficiently large n , on every random execution. The stated intermediate form is $2^{0.489995n} \text{poly}(n + b + 2)$.

Table 1: Corrected status of the main classical regimes.

Setting		Bound	Status
Positive-integer Sum, polynomial-bit RAM	Subset word	$2^{0.49n}$	claimed in a new preprint
Signed-integer with an occurrence witness	Subset Sum	$2^{n/2}$ baseline	no reduction or implementation here
Random hard knapsacks		$2^{0.291n}$	BCJ model; average-case
Pigeonhole Equal Sum	Subset	$2^{n/3}$	randomized promise result

The claim is therefore a substantial change to the worst-case landscape, if correct. It is narrower than the signed-integer, witness-returning interface of this repository: the manuscript states a positive-integer decision theorem. An explicit reduction and witness reconstruction must be supplied before its bound can be assigned to the repository API.

3 Pipeline in the manuscript

The new proof is not a direct extension of the HGJ or BCJ representation algorithm. Its five coupled components are:

1. **Isolation.** Random tie-breaking weights transform one input into polynomially many positive-weight targets. With high constant probability, a satisfiable input produces a target with exactly one representing subset.
2. **Preliminary search.** A capped MITM procedure solves instances whose random restrictions have a sufficiently small distinct subset-sum support.
3. **Modular checksum.** For a large random prime, a filtered high/low-digit construction estimates a Fourier expression for each modular target bin to small *absolute* error. The filter uses fixed numerical parameters and a finite interval-arithmetic certificate in the appendix.
4. **Alias extraction.** A separate shared-block representation lists modular false positives. Its records carry masks for the shared block; a pair is compatible exactly when those masks are disjoint. Random primes bound surviving nonzero integer discrepancies.
5. **Subtraction and thresholding.** The estimated modular bin minus the complete alias contribution distinguishes an empty bin from the isolated one.

This structure matters for reproducibility. The claimed exponent depends on the interaction of checksum variance, filtering coverage, alias enumeration, finite precision, and caps on every randomized loop. Replacing any one part by an uncapped heuristic does not reproduce Theorem 1.1.

4 Audit and repository status

The exact manuscript path, theorem scope, and published model were checked against the primary PDF. The official Lean formalization catalog does not list this manuscript [3]. The collection

release also says that results are at different verification stages and that not all manuscripts are formalized [2]. Accordingly, this repository labels the result *unreviewed until independently checked*; that label concerns the state of this project’s audit, not a claim that the manuscript is false.

The former Python small-instance model was removed in the Lean-first migration. The current checked executable pipeline covers the proved reference meet-in-the-middle decision procedure only. A Lean formalization of the isolation family, shared-mask compatibility, estimator, filter-table construction, alias sampler, arithmetic certificate, and word-RAM engineering remains necessary before any end-to-end claim about this preprint.

The historical reports record earlier three-block and PESS experiments. The Lean project retains a theorem for the three-block sum decomposition. Neither supplies the missing end-to-end implementation of the new manuscript.

5 Revised next steps

1. Reproduce the modular-checksum and alias-extraction theorems on small instances with explicit failure caps and a brute-force oracle.
2. Independently check the filtering-moment inequalities and the rational interval calculations in the appendix.
3. Specify and prove a positive-to-signed reduction that preserves the repository’s witness semantics.
4. Only after those steps, update the main survey’s worst-case table and conclusion from “open” to a properly qualified account of the preprint.

References

- [1] OpenAI, “Subset Sum in Time $\mathcal{O}(2^{0.49n})$,” October 4, 2026. <https://github.com/openai/math/blob/main/preprints/Subset-Sum-in-Time-2-power-0-49n-October-4-2026/subset-sum.pdf>
- [2] OpenAI, “Sharing AI progress in mathematics,” October 6, 2026. <https://openai.com/index/sharing-ai-progress-in-mathematics/>
- [3] OpenAI, “Lean formalization catalog,” accessed October 9, 2026. <https://github.com/openai/math/blob/main/lean/formalization.yaml>
- [4] A. Becker, J.-S. Coron, and A. Joux, “Improved generic algorithms for hard knapsacks,” EU-ROCRYPT 2011. <https://eprint.iacr.org/2011/474>
- [5] C. Jin, R. Williams, and S. Zhang, “New algorithms for Pigeonhole Equal Subset Sum,” ESA 2025. <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ESA.2025.86>